MODULE 11 ·

What Django REST Framework adds

DRF is Django’s toolkit for turning application logic and models into usable Web APIs.

Django **Model** → DRF **API** → Client **JSON**

Serializers | APIView | Request / Response | Authentication | Browsable API | Validation

The API becomes a clear boundary between the Django application and the clients that consume its data.

SLIDE 12 · WORKSPACE

Set up the Book API workspace

```
python -m venv venv
source venv/bin/activate
# Windows
venv\Scripts\activate
pip install django djangorestframework
```

01 **Isolate**

Create a project-only Python environment.

02 **Install**

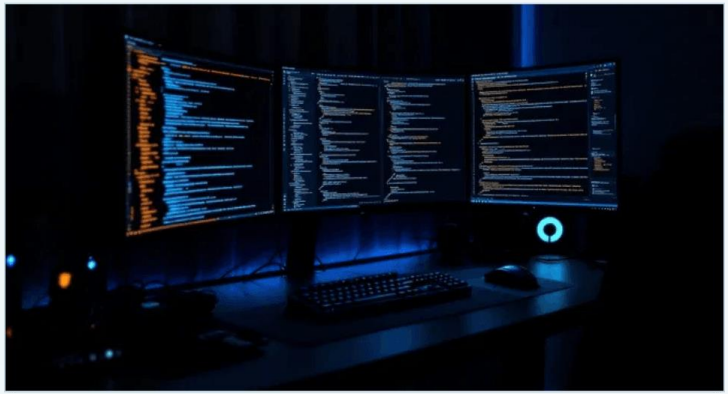
Add Django and the DRF toolkit.

03 **Scaffold**

`startproject config .` creates project configuration.

04 **Organize**

`startapp books` creates the API domain.



A clean environment makes the first API loop easier to debug.

SLIDE 13 · CONFIGURATION

Register DRF before writing API code

CONFIG/SETTINGS.PY

```
INSTALLED_APPS = [
...
"rest_framework",
"books",
]
```

SYNC + SERVE

```
python manage.py migrate
python manage.py runserver
```

01 **Activate DRF**

`rest_framework` makes the framework available to Django.

02 **Discover the app**

`books` exposes the model and migrations to the project.

03 **Build the database**

`migrate` creates Django's built-in tables.

04 **Open the loop**

`runserver` gives us a local target to test.

Quick fix: "No module named rest_framework" usually means the virtual environment is inactive or the package was installed elsewhere.

Model the resource before exposing it

```
# books/models.py
from django.db import models
 class Book(models.Model):
title = models.CharField(max_length=200)
author = models.CharField(max_length=100)
published_year = models.PositiveIntegerField()
```

1. Book Model

The source of truth defining the resource structure.

2. Database Rows

Created via `makemigrations` and `migrate` .

3. API Representations

Data exposed to clients, mapped from the model.

Serializers are the API translation layer

A serializer translates between Python/Django objects and JSON-
JSON-compatible data. It acts as a strict two-way gatekeeper for your API.
your API.

Model → Client

Convert complex model objects and querysets into JSON-compatible
compatible output for the client.

Validation

Inspect and validate incoming request data before allowing it to be saved
be saved to the database.

Client → Model

Turn validated JSON data back into fully-formed Django model instances.
instances.

Let ModelSerializer generate the repetitive parts

```
from rest_framework import serializers from .models
import Book class
BookSerializer(serializers.ModelSerializer): class Meta:
model = Book fields = "__all__"
```

ModelSerializer Shortcut

Automatically generates fields and default create/update logic based on the model definition.

Model Mapping

The `model = Book` declaration connects the serializer directly to your database schema.

Explicit Fields Best Practice

While `"__all__"` is fast for learning, production APIs should explicitly list fields to prevent accidental data exposure.

One APIView can support GET and POST

```
from rest_framework.views import APIView;
from rest_framework.response import Response;
from rest_framework import status
from .models import Book;
from .serializers import BookSerializer
class BookListCreateView(APIView):
def get(self, request):
books = Book.objects.all()
serializer = BookSerializer(books, many=True)
return Response(serializer.data)
   def post(self, request):
serializer = BookSerializer(data=request.data)
if serializer.is_valid():
serializer.save();
return Response(serializer.data, status=status.HTTP_201_CREATED)
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

`many=True`

Tells the serializer to process a queryset (multiple objects) rather than a single instance.

`request.data`

Reads the client's parsed JSON body.

`.is_valid()`

Validates before save; populates `.errors` on failure.

Route the view to a predictable endpoint

```python
# books/urls.py
from django.urls import path
from .views import BookListCreateView
 urlpatterns = [
path("books/", BookListCreateView.as_view()),
]
```

```python
# config/urls.py
from django.urls import include, path
 urlpatterns = [
path("api/", include("books.urls")),
]
```

The as_view() Bridge

Converts the class-based `APIView` into a function that Django's URL router can call.

Final Endpoint: `/api/books/`

The project contributes the `/api/` prefix, and the app contributes the `books/` path.

Test the contract, not just the code

```
GET http://127.0.0.1:8000/api/books/
 POST http://127.0.0.1:8000/api/books/
{
"title": "Django REST APIs",
"author": "A. Developer",
"published_year": 2026
}
```

200 OK — **Read succeeded**

GET returns a list of resources.

201 Created — **Resource created**

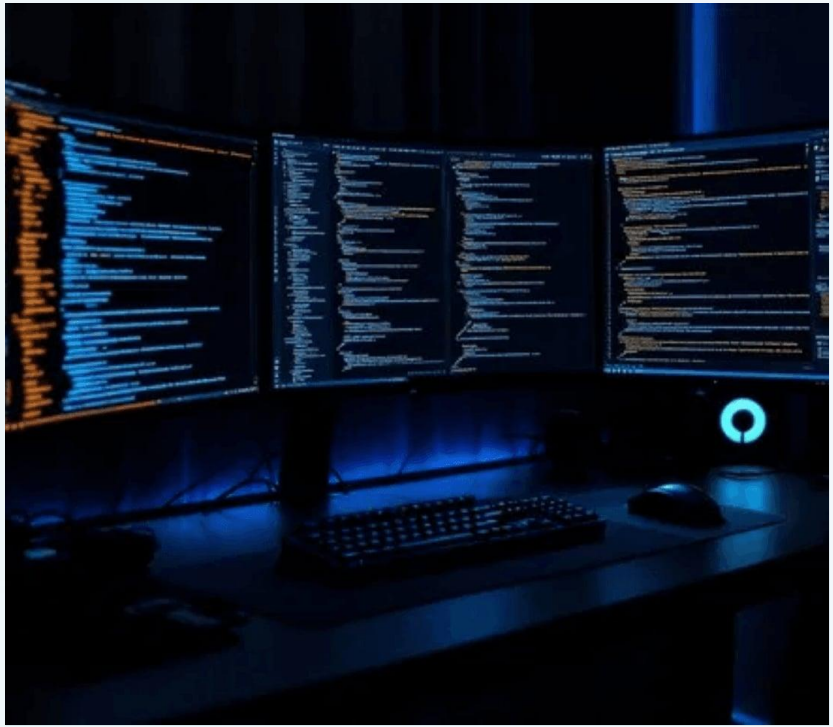
POST passes validation and saves data.

400 Bad Request — **Input is invalid**

Missing or malformed field in the request body.

404 Not Found — **Route does not exist**

Wrong URL or missing prefix.



Recap, then build a Student API

THE API LEARNING PATH

REST → JSON → HTTP → DRF Setup → Serializer → APIView → URL Routing → Testing

1. Model & Migrate

Create a `Student` model (name, email, course, age) and run migrations.

2. Serialize & Route

Create a `StudentSerializer`, build GET/POST views, and configure URLs.

BEGINNER MISTAKE CHECKLIST

- ☐ Forgot to add app to `INSTALLED_APPS`?
- ☐ Ran `migrate` but not `makemigrations`?
- ☐ Forgot `many=True` for a queryset?
- ☐ Called `.save()` before `.is_valid()`?